Object Oriented Programming with Java

COMSM0086

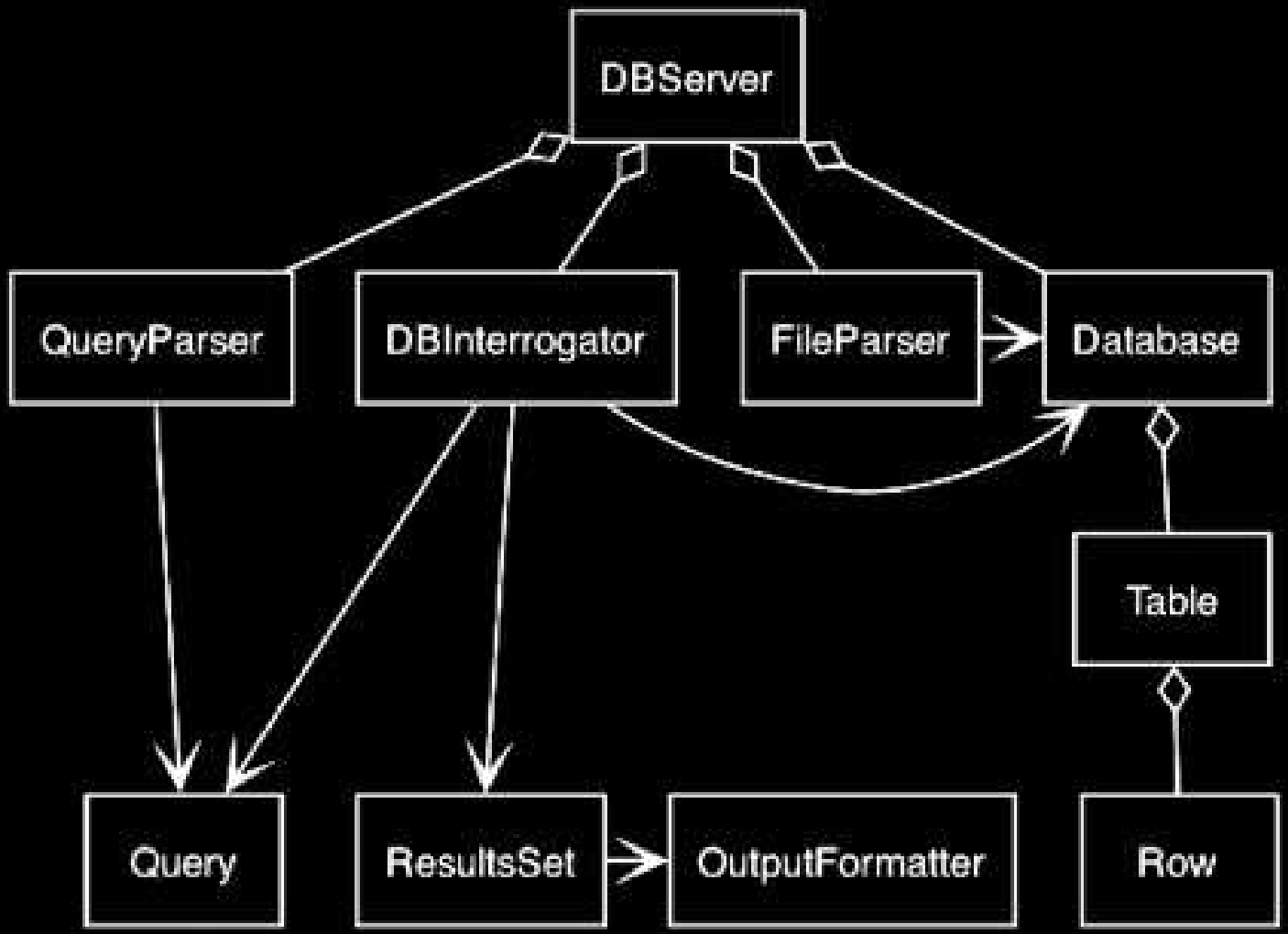
DB Exercise Debriefing

Looking for the "Correct" Solution ?

There isn't really one single "correct" solution
But there are "good" and "not-so-good" approaches

This notion of "goodness" is a slippery concept
Open to interpretation, but typically involves:

- A reasonable (logical/extensible) structure
- Appropriate material code quality attributes
- Behaves as intended (passes the tests !)
- Operates "efficiently enough" to be usable

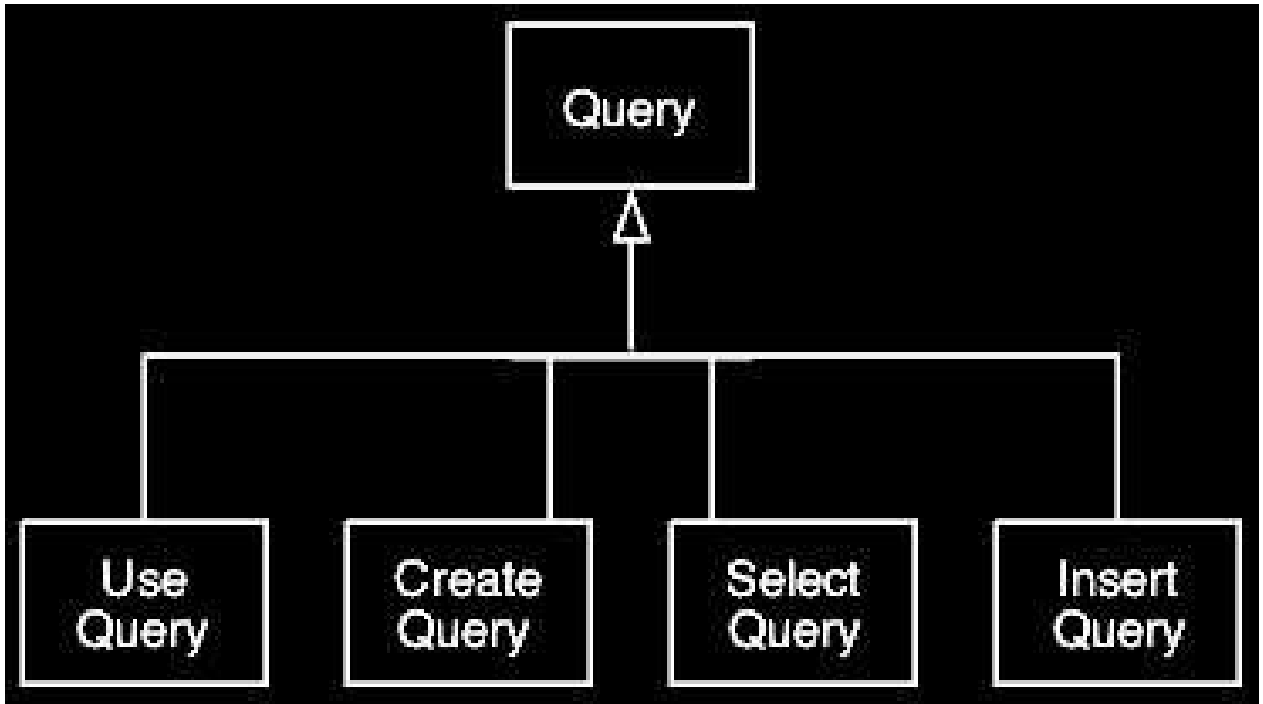

DBServer
QueryParser
DBInterrogator
FileParser
Database
Table
Query
ResultsSet
OutputFormatter
Row

Query Inheritance Hierarchy

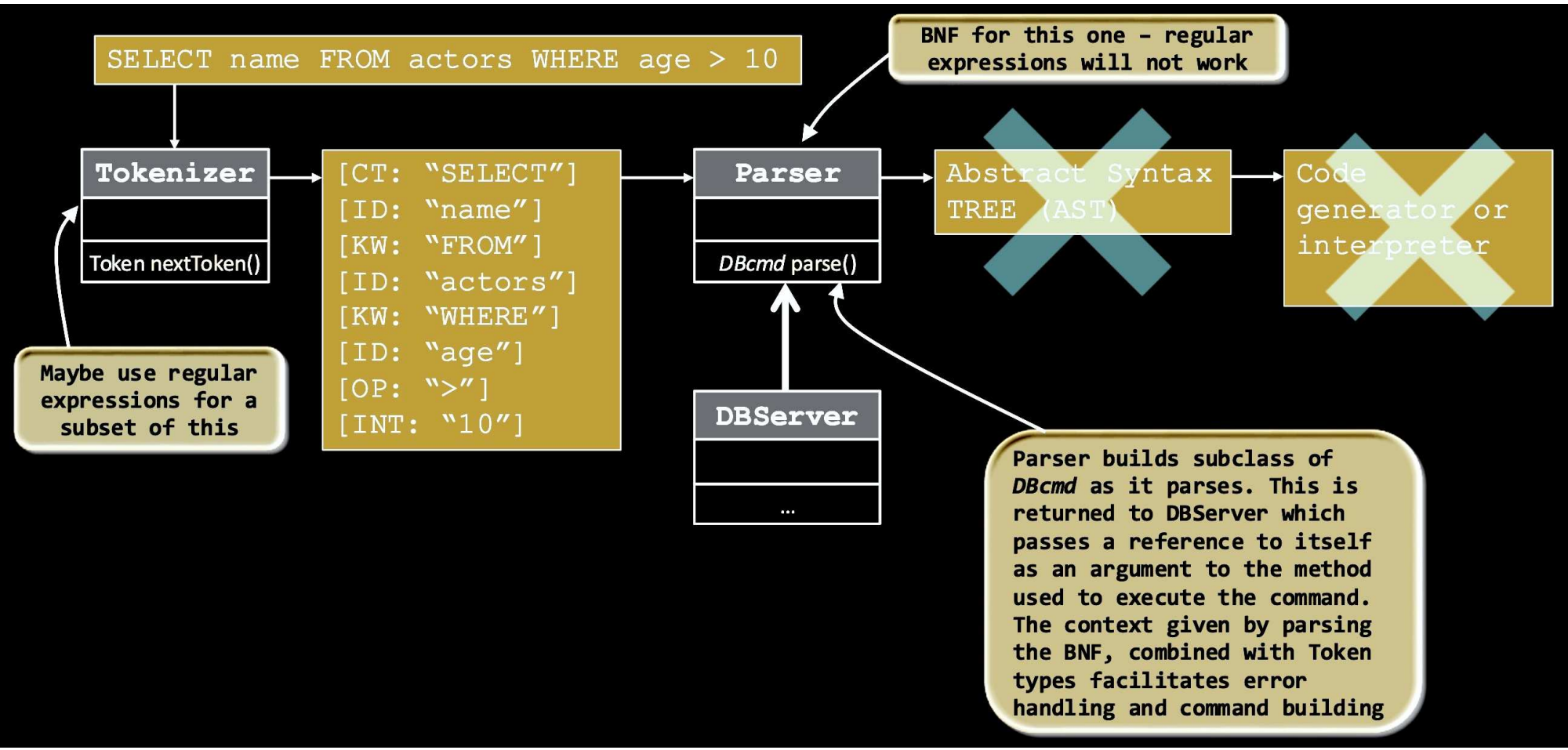
All the possible query type have their own class

All query types inherit from a abstract super class

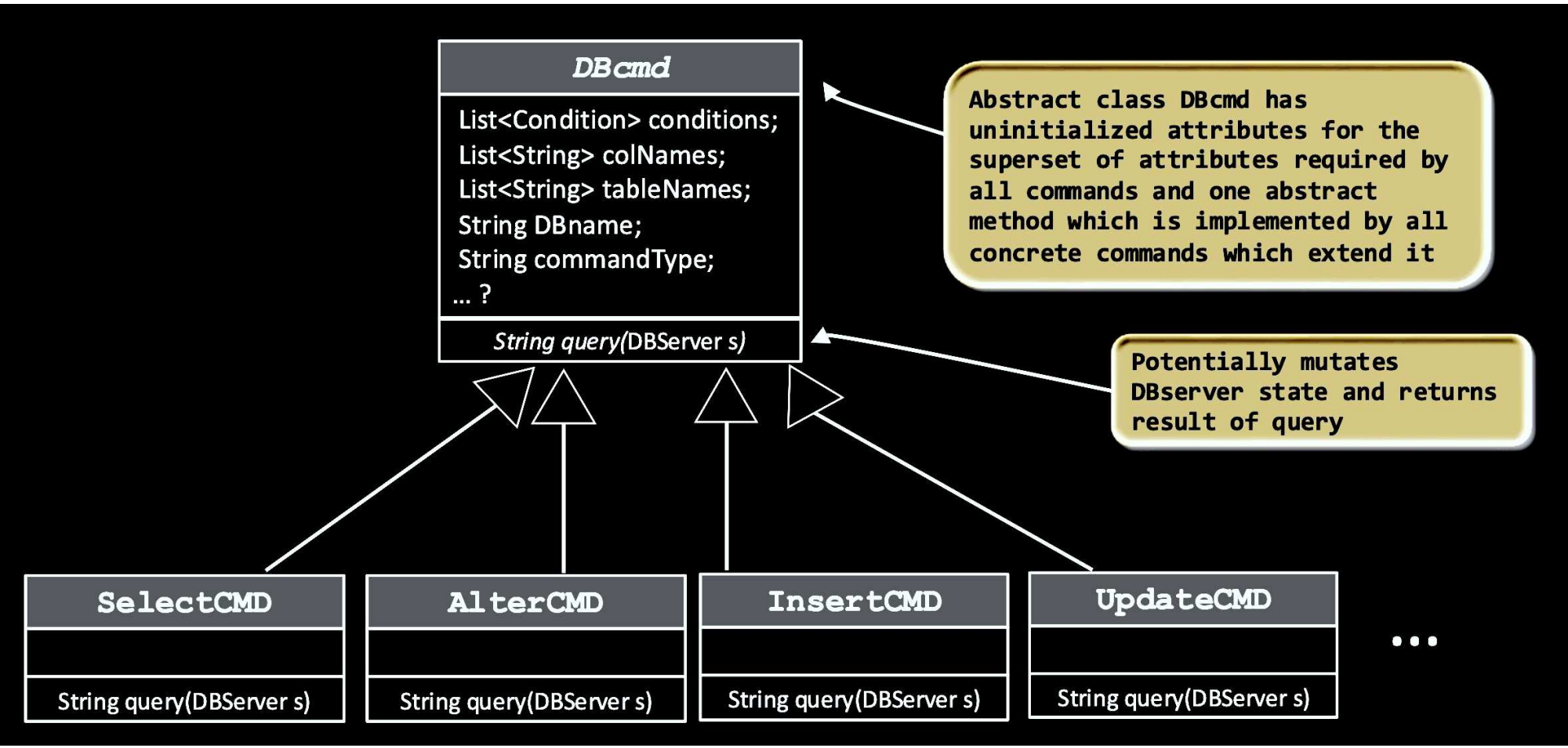
Each class can parse and perform that query type



Sion's Preferred Architecture



Sion's Preferred Architecture



Code Quality Feedback

We have set up a submission point on Blackboard
If upload your DB project code here by Friday 1pm
You'll receive an email summary of quality analysis

Report highlights ALL issues detected in your code
(Some points raised are more serious than others)

It's useful however to be made aware of all issues
The best way to continuously improve your code !

Most commonly encountered issues

- Overly complex methods, as measured by:
  - Cyclomatic Complexity (see quality lecture)
  - Depth of Nesting (indentation)
  - Sheer length of methods (Lines of Code)
- Replication and Duplication (unDRY code)
- Breaking encapsulation (public attributes)
- Unnecessarily long IF/CASE statement blocks

Testing: "Behaves as intended"

Our test cases cover as much behaviour as possible

Whilst trying to keep the test set small and tight

Our test cases are split into the following "clusters":

- Basic & Compound SELECT
- UPDATE, DELETE and JOIN
- Illegal Names & Illegal Actions
- Malformed Queries & Unknown Entities
- Case insensitivity & Whitespace Variability
- General Server Robustness (miscellaneous!)

Run Our Test Cases

We will release all of our test cases via GitHub
(When we feel that the time is right !)

Run them against your code, see how many you pass
If you fail our tests, your own cases were incomplete

Ask why you overlooked those areas of behaviour ?
How might you improve your own tests in future ?
Could you be more systematic in your approach ?

Efficiency

Efficiency was not explicitly part of this exercise
Hard to assess without employing large datasets
Even then, devising suitable metrics is problematic
(Hard to determine what java is doing internally !)

This topic is not covered to any extent in this unit
(Not really enough time to consider algorithmics)
Something to bare in mind when designing solutions
Especially important creating non-trivial systems

Importance of this Discussion

At the very end of this unit there will be...
A BIG assessed exercise (70% of the unit mark)

It is important to gain feedback on your work NOW
(so you can reflect on your performance and improve)

You don't really want the first feedback you get...
To be AFTER the one (and only) assessed exercise !